

阿里

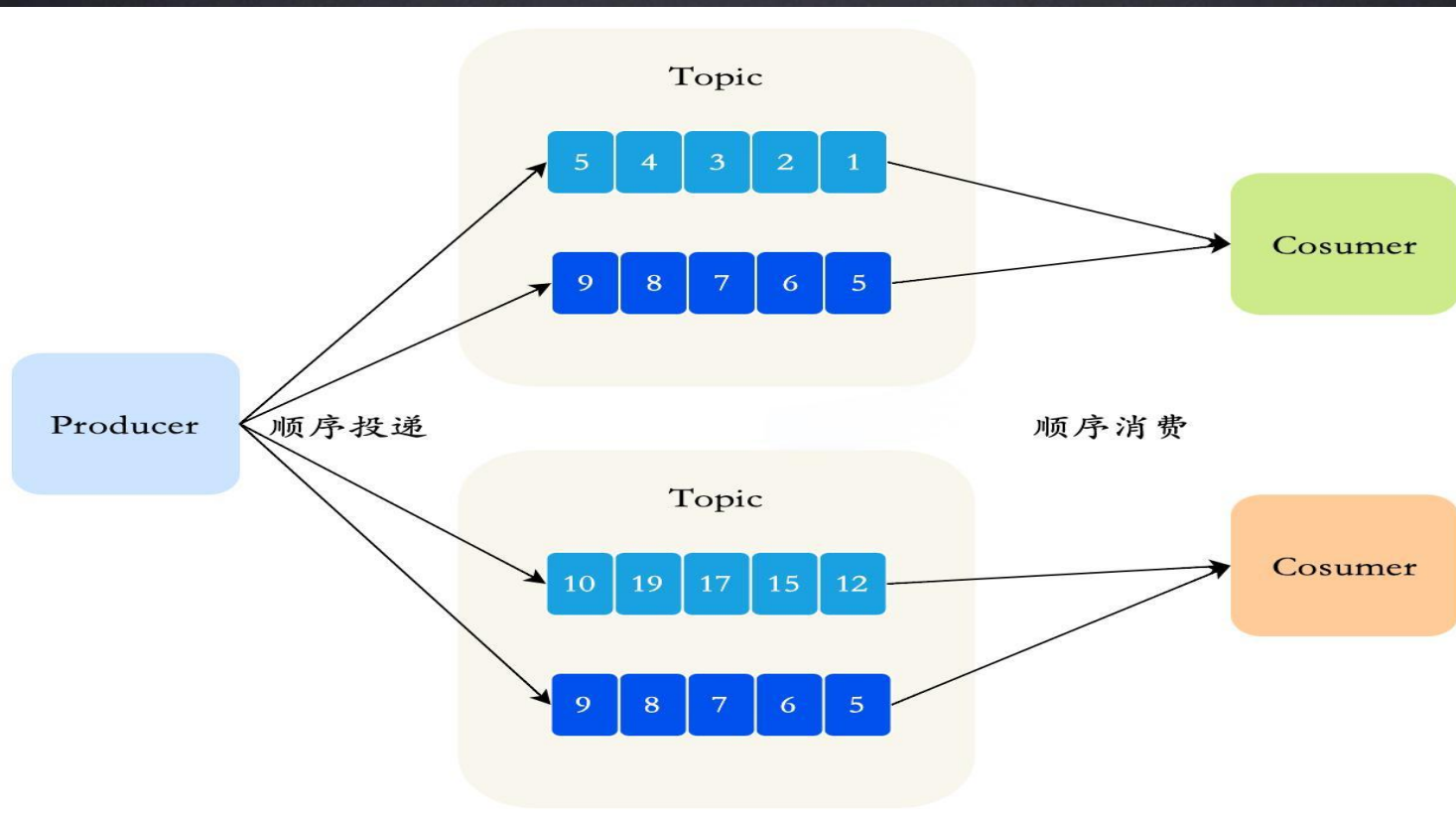
如何确保消息的顺序性？

🔥 薪资：35K

🌟 岗位：高级开发

顺序消息的概念

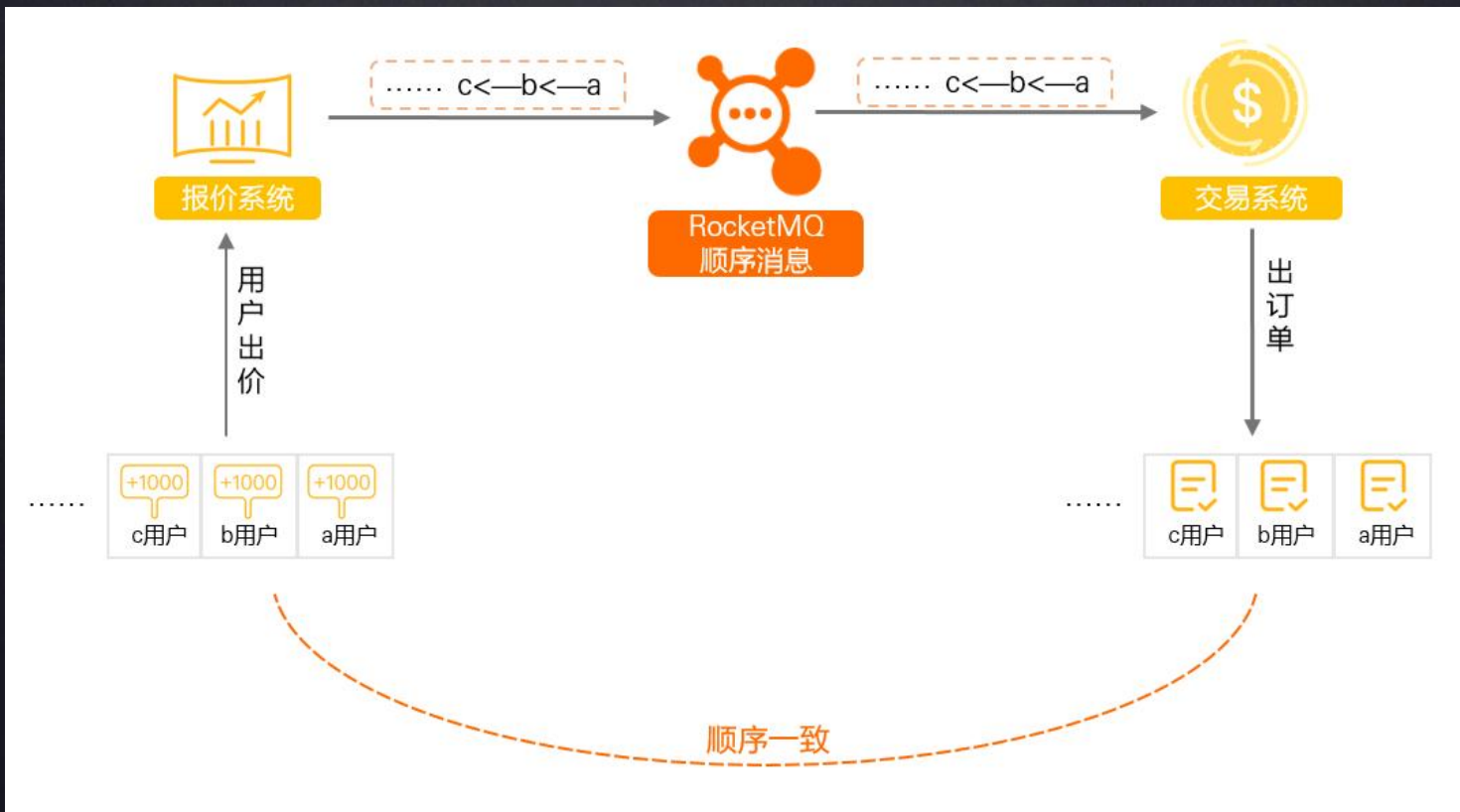
顺序消息是一种对消息发送和消费顺序有严格要求的消息。对于一个指定的Topic，消息严格按照先进先出（FIFO）的原则进行消息发布和消费，即先发布的消息先消费，后发布的消息后消费，顺序消息常用于金融证券、电商业务等对消息指令顺序有严格要求的场景



顺序消息的场景

典型场景一：撮合交易

以证券、股票交易撮合场景为例，对于出价相同的交易单，坚持按照先出价先交易的原则，下游处理订单的系统需要严格按照出价顺序来处理订单。



顺序消息的场景

典型场景二：数据实时增量同步

以数据库变更增量同步场景为例，上游源端数据库按需执行增删改操作，将二进制操作日志作为消息，通过RocketMQ传输到下游搜索系统，下游系统按顺序还原消息数据，实现状态数据按序刷新。如果是普通消息则可能会导致状态混乱，和预期操作结果不符，基于顺序消息可以实现下游状态和上游操作结果一致。

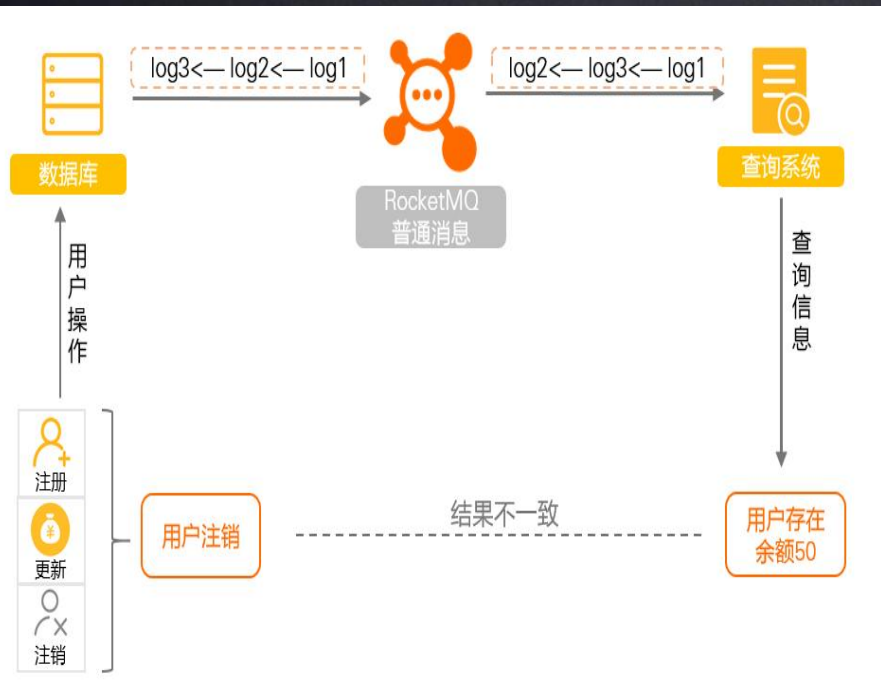


图 1. 普通消息

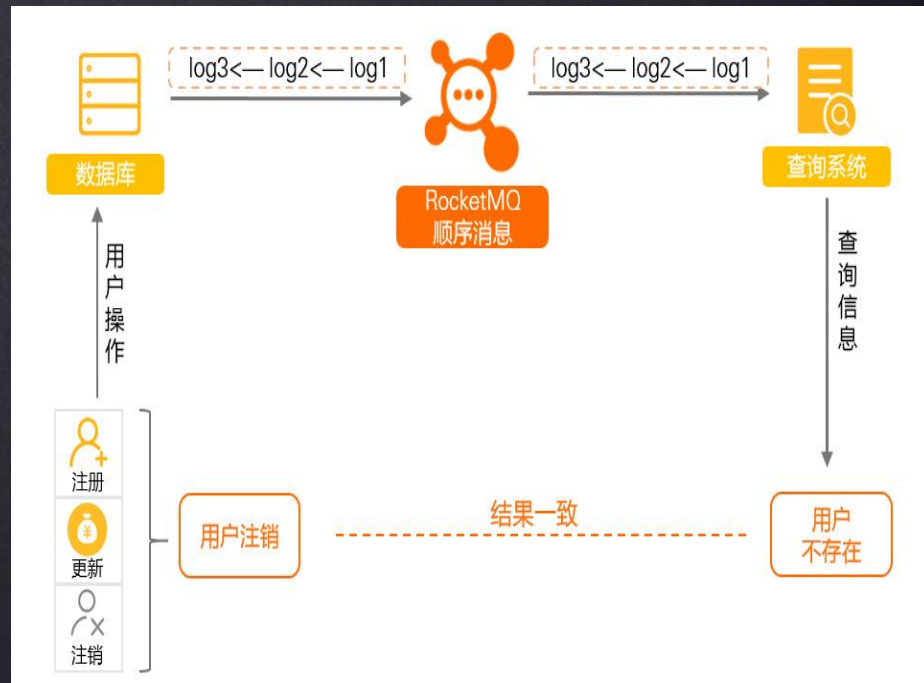


图 2. 顺序消息

顺序消息类型

顺序消息分为全局顺序消息和分区顺序消息。

全局顺序消息：指对于指定的一个Topic，所有消息按照严格的先入先出（FIFO）的顺序来发布和消费。

适用场景：适用于性能要求不高，所有的消息严格按照FIFO原则来发布和消费的场景。

示例：在证券处理中，以人民币兑换美元为Topic，在价格相同的情况下，先出价者优先处理，则可以按照FIFO的方式发布和消费全局顺序消息。

分区顺序消息：对于指定的一个Topic，所有消息根据Sharding Key进行区块分区，同一个分区内的消息按照严格的先进先出（FIFO）原则进行发布和消费。同一分区内的消息保证顺序，不同分区之间的消息顺序不做要求。

适用场景：适用于性能要求高，以Sharding Key作为分区字段，在同一个区块中严格地按照先进先出（FIFO）原则进行消息发布和消费的场景。

示例：

- 用户注册需要发送验证码，以用户ID作为Sharding Key，那么同一个用户发送的消息都会按照发布的先后顺序来消费。
- 电商的订单创建，以订单ID作为Sharding Key，那么同一个订单相关的创建订单、订单支付、订单退款、订单物流等消息都会按照发布的先后顺序来消费。
- 阿里内部电商系统均使用分区顺序消息，既保证业务的顺序，同时又能保证业务的高性能。

说明：全局顺序消息实际上是一种特殊的分区顺序消息，即Topic中只有一个分区，因此全局顺序和分区顺序的实现原理相同。因为分区顺序消息有多个分区，所以分区顺序消息比全局顺序消息的并发度和性能更高。

如何保证消息的顺序性

RocketMQ 的消息的顺序性分为两部分，**生产顺序性**和**消费顺序性**。

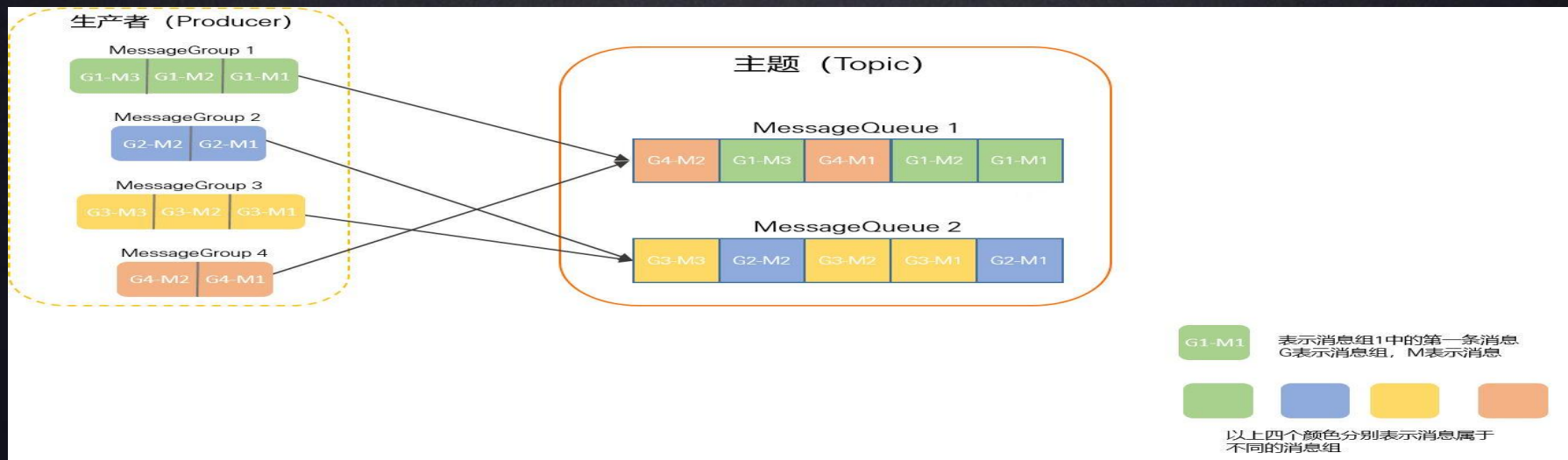
生产顺序性：RocketMQ 通过生产者和服务端的协议保障单个生产者串行地发送消息，并按序存储和持久化。

同一消息组（MessageGroup）：消息生产顺序性的范围为消息组，生产者发送消息时可以为每条消息设置消息组，只有同一消息组内的消息可以保证顺序性，不同消息组或未设置消息组的消息之间不保证顺序。

- **单一生产者**：消息生产的顺序性仅支持单一生产者，不同生产者分布在不同的系统，即使设置相同的消息组，不同生产者之间产生的消息也无法判定其先后顺序。
- **串行发送**：RocketMQ生产者客户端支持多线程安全访问，但如果生产者使用多线程并行发送，则不同线程间产生的消息将无法判定其先后顺序。

满足以上条件的生产者，将顺序消息发送至RocketMQ后，会保证设置了同一消息组的消息，按照发送顺序存储在同一队列中。服务端顺序存储逻辑如下图

- 相同消息组的消息按照先后顺序被存储在同一个队列。
- 不同消息组的消息可以混合在同一个队列中，且不保证连续。如下图所示，消息组1和消息组4的消息混合存储在队列1中，RocketMQ 保证消息组1中的消息G1-M1、G1-M2、G1-M3是按发送顺序存储，且消息组4的消息G4-M1、G4-M2也是按顺序存储，但消息组1和消息组4中的消息不涉及顺序关系。



如何保证消息的顺序性

消费顺序性：RocketMQ通过消费者和服务端的协议保障消息消费严格按照存储的先后顺序来处理。

如需保证消息消费的顺序性，则必须满足以下条件：

- **投递顺序：**RocketMQ通过客户端SDK和服务端通信协议保障消息按照服务端存储顺序投递，但业务方消费消息时需要严格按照接收—处理—应答的语义处理消息，避免因异步处理导致消息乱序。
- **有限重试：**RocketMQ顺序消息投递仅在重试次数限定范围内，即一条消息如果一直重试失败，超过最大重试次数后将不再重试，跳过这条消息消费，不会一直阻塞后续消息处理。对于需要严格保证消费顺序的场景，请务必设置合理的重试次数，避免参数不合理导致消息乱序。

顺序消息的使用建议

串行消费，避免批量消费导致乱序，消息消费建议串行处理，避免一次消费多条消息，否则可能出现乱序情况。

例如：发送顺序为1->2->3->4，消费时批量消费，消费顺序为1->23（批量处理，失败）->23（重试处理）->4，此时可能由于消息3的失败导致消息2被重复处理，最后导致消息消费乱序。

和普通消息发送相比，顺序消息发送必须要**设置消息组**。消息组的粒度建议按照业务场景，尽可能**细粒度设计**，以便实现业务拆分和并发扩展。**消息组尽可能打散**，避免集中导致热点，RocketMQ 保证相同消息组的消息存储在同一个队列中，如果不同业务场景的消息都集中在少量或一个消息组中，则这些消息存储压力都会集中到服务端的少量队列或一个队列中。容易导致性能热点，且不利于扩展。一般建议的**消息组设计会采用订单ID、用户ID作为顺序参考**，即同一个终端用户的消息保证顺序，**不同用户的消息无需保证顺序**。因此建议将业务以消息组粒度进行拆分，例如，将订单ID、用户ID作为消息组关键字，可实现同一终端用户的消息按照顺序处理，不同用户的消息无需保证顺序。

总结

1. 顺序消息是一种对消息发送和消费顺序有严格要求的消息。对于一个指定的Topic，消息严格按照先进先出（FIFO）的原则进行消息发布和消费，即**先发布的消息先消费，后发布的消息后消费**
2. 顺序消息的典型场景：撮合交易、数据实时增量同步
3. 顺序消息的类型：**全局顺序消息**和**分区顺序消息**
 - ◆ 全局顺序消息：指对于指定的一个Topic，所有消息按照严格的先入先出（FIFO）的顺序来发布和消费。
 - ◆ 分区顺序消息：对于指定的一个Topic，所有消息根据Sharding Key进行区块分区，同一个分区内的消息按照严格的先进先出（FIFO）原则进行发布和费。同一分区内的消息保证顺序，不同分区之间的消息顺序不做要求。
4. 消息的顺序性可以分为：**生产顺序性**和**消费顺序性**
 - ◆ 生产顺序性：RocketMQ 通过生产者和服务端的协议保障单个生产者串行地发送消息，并按序存储和持久化。
 - ◆ 消费顺序性：RocketMQ通过消费者和服务端的协议保障消息消费严格按照存储的先后顺序来处理。
5. 顺序消息的使用建议
 - ◆ 消息消费建议串行处理，避免一次消费多条消息，否则可能出现乱序情况。
 - ◆ 消息组尽可能打散，避免集中导致热点